Assignment 2

Report

Processor: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
Physical cores: 4 (Total logical CPUs: 8)
Operating System: Ubuntu 24.04.3 LTS (Codename: noble)
GCC version: 13.3.0

Name: Pavlos Tomazos **ID:** 1115202200188

Exercise 2.1

This exercise addresses the same problem as Exercise 1.1 from the previous assignment. The main difference is that the implementation is changed from using `pthreads` to using `OpenMP`. As expected, the OpenMP version results in a cleaner and more elegant code structure, since there is no need to explicitly compute the total workload and manually divide it among threads. Instead, OpenMP's `schedule(static,1)` (cyclic scheduling) is used, which automatically balances the workload across threads. The scheme is illustrated below:

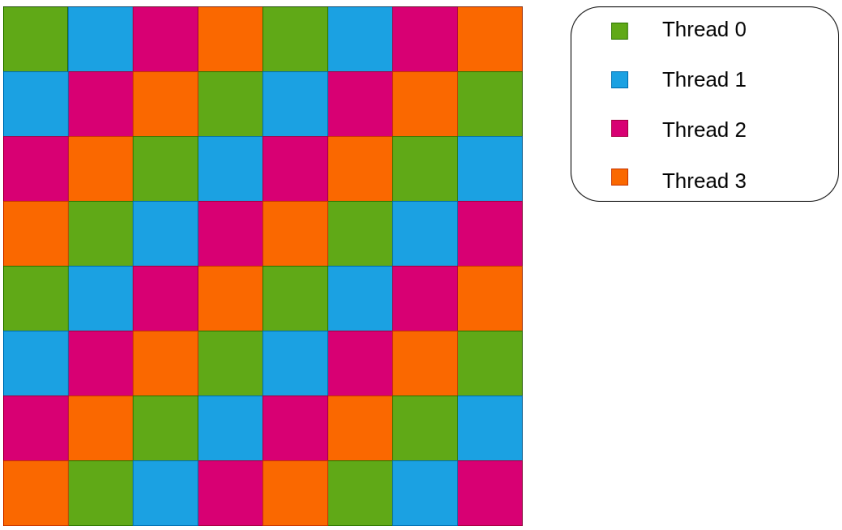


Figure 1: Array representation of the double loop and cyclic division among threads

Overall, the OpenMP implementation is significantly more readable compared to the `pthreads` version.

Algorithm 1 Naive Parallel Polynomial Multiplication (Cyclic Scheduling)

Require: Polynomials a, b of degree n, number of threads T
Ensure: Polynomial $result$ of degree $2n$

```
result ← new polynomial of size 2n + 1
for k = 0 to 2n with cyclic scheduling using T threads do
    sum ← 0
    i_start ← max(0, k − n)
    i_end ← min(k, n)
    for i = i_start to i_end do
        sum ← sum + a[i] · b[k − i]
    end for
    result[k] ← sum
end for
return result
```

Analysis of Results

The performance scaling follows a similar trend to the `pthreads` implementation. However, the OpenMP version achieves slightly better execution times, mainly due to the absence of overhead related to manually computing and distributing the workload.

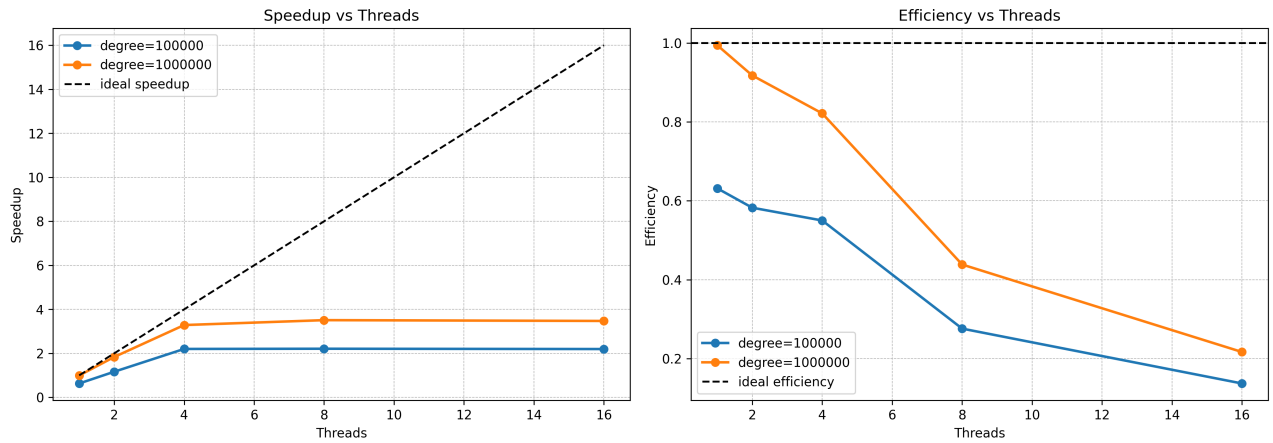


Figure 2: Parallel efficiency and speedup of the OpenMP implementation

In summary, the OpenMP approach provides a more concise and maintainable solution, while also offering modest performance improvements.

Exercise 2.2

The objective of this exercise is to compare two methods for array–vector multiplication for sparse matrices, where zeros dominate. To store only the non-zero elements, the **Compressed Sparse Row (CSR)** format uses three arrays:

- `values`: non-zero matrix elements in row-major order
- `column_indices`: column index for each value
- `row_pointers`: starting index of each row in `values`

This format enables sparse matrix–vector multiplication by iterating exclusively over non-zero values, avoiding unnecessary operations on zeros.

This exercise investigates parallelism at multiple levels of the implementation. First, the conversion from a dense matrix to CSR format—referred to as CSR build—is performed. The pseudocode below illustrates CSR build and highlights the sections that can be parallelized using OpenMP.

Algorithm 2 CSR Build (Serial / Parallel)

```
Initialize row_nnz[0..N-1], nnz ← 0
for all i = 0 to N − 1 (parallel possible) do
    row_nnz[i] ← count of non-zero elements in row i
    nnz ← nnz + row_nnz[i]
end for
Compute prefix sum of row_nnz → row_pointers
Allocate values and column_indices of size nnz
for all i = 0 to N − 1 (parallel possible) do
    for each non-zero element A[i][j] in row i do
        Store A[i][j] in values at row_pointers[i]
        Store j in column_indices at same position
    end for
end for
```

The next step is performing matrix–vector multiplication using the CSR format. The pseudocode below shows the CSR mat–vec operation, highlighting the parts that can be parallelized using OpenMP.

Algorithm 3 CSR Mat–Vec (Serial / Parallel)

```
Initialize r[0..N − 1] ← 0
for all i = 0 to N − 1 (parallel possible) do
    sum ← 0
    for j = row_pointers[i] to row_pointers[i + 1] − 1 do
        sum ← sum + values[j] × v[column_indices[j]]
    end for
    r[i] ← sum
end for
```

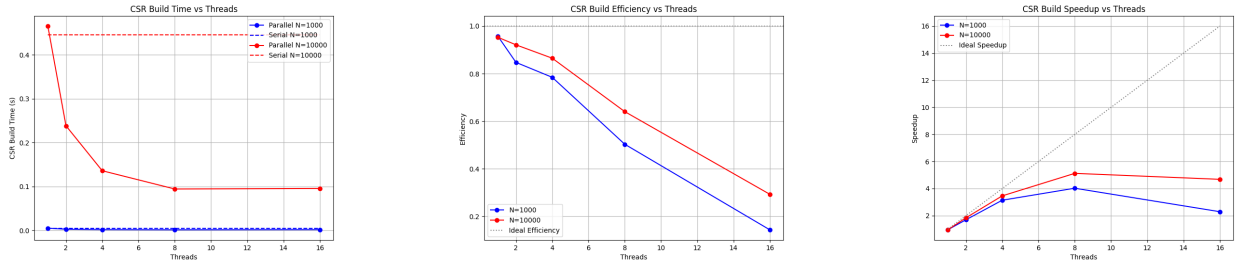
Finally, multiplication using the dense matrix representation is presented for completeness. This will be used later to compare results with the sparse implementation.

Algorithm 4 Dense Mat–Vec (Serial / Parallel)

```
Initialize r[0..N − 1] ← 0
for all i = 0 to N − 1 (parallel possible) do
    sum ← 0
    for j = 0 to N − 1 do
        sum ← sum + A[i][j] × v[j]
    end for
    r[i] ← sum
end for
```

Analysis of Results

First, we examine the parallelism of the CSR build. For this analysis, parameters such as sparsity and the number of multiplication iterations are ignored. The primary factors affecting the evaluation of the parallel implementation are the data size ($N \times N$) and the number of threads used. The figure below presents the execution times, efficiency, and speedup of the parallel implementation, using the serial implementation as a baseline.

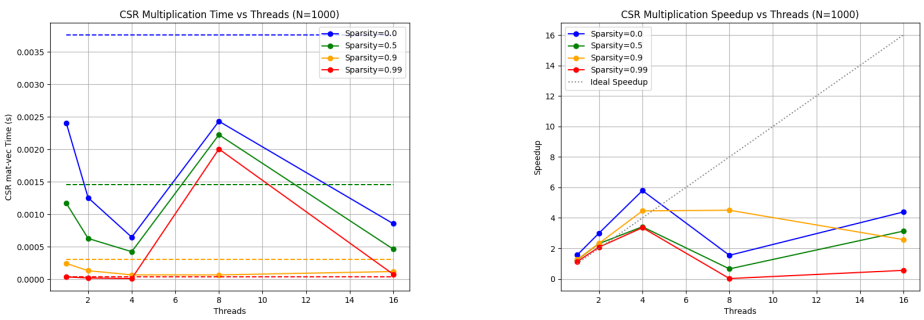


(a) CSR build execution time (b) CSR build efficiency (c) CSR build speedup

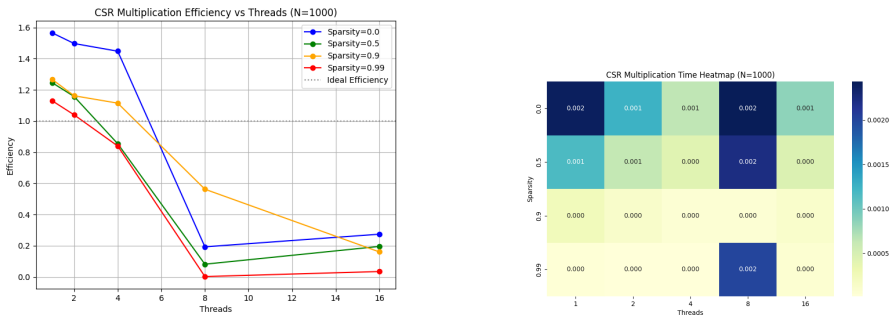
Figure 3: Comparison of CSR build serial and parallel approach

The parallel CSR build reduces execution time, with greater improvements for larger data sizes, showing good scalability. Performance peaks at 4 threads, beyond which additional threads provide little benefit. Efficiency (over 80%) and speedup near to ideal ($x4$) confirm that 4 threads is the optimal configuration.

Next, we analyze the parallel performance of CSR multiplication. Sparsity plays a significant role and is a key factor in interpreting the results.

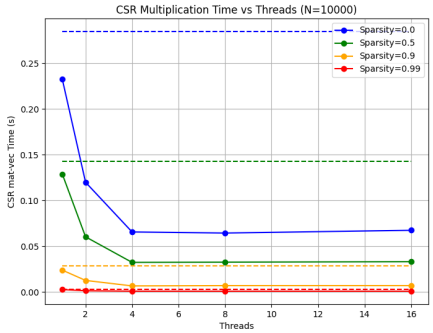


(a) CSR Multiplication Time (N=1000) (b) Speedup vs Threads (N=1000)

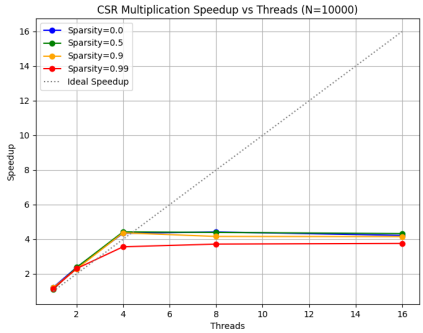


(c) Efficiency vs Threads (N=1000) (d) Time Heatmap (N=1000)

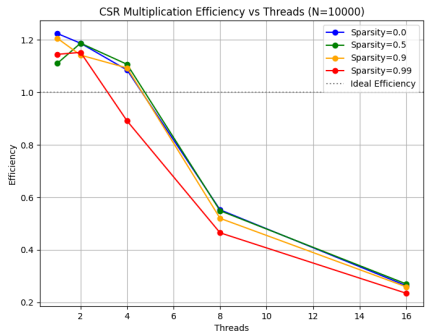
Figure 4: CSR Multiplication Benchmark Results for $N = 1000$. Top-left: Time, top-right: Speedup, bottom-left: Efficiency, bottom-right: Heatmap.



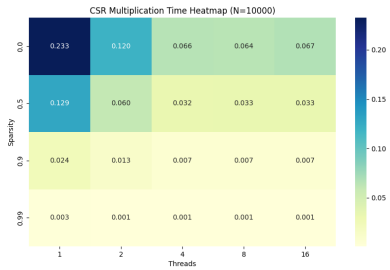
(a) CSR Multiplication Time (N=10000)



(b) Speedup vs Threads (N=10000)



(c) Efficiency vs Threads (N=10000)



(d) Time Heatmap (N=10000)

Figure 5: CSR Multiplication Benchmark Results for N=10000. Layout same as the previous page.

The figures above illustrate the CSR parallel multiplication time compared to the serial implementation for $N = 10^3$ and $N = 10^4$. The trends are consistent across both problem sizes, with the larger scale providing a clearer view:

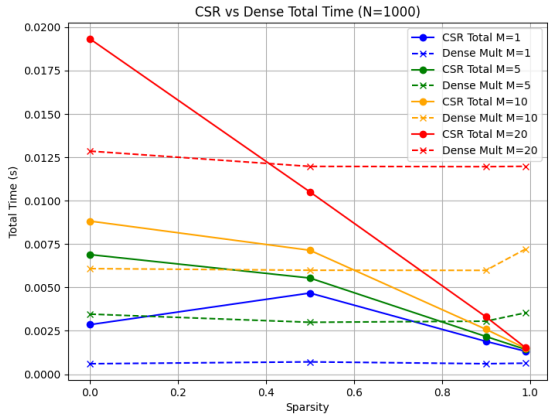
- Execution time decreases as sparsity increases, for both serial and parallel implementations.
- With higher sparsity, the CSR approach benefits more, and the impact of parallelism becomes relatively smaller.
- For all sparsity levels, using 4 threads achieves nearly ideal speedup, slightly exceeding the theoretical value.
- Similarly, efficiency is close to ideal and slightly above 100%.
- Heatmaps indicate that execution times are lowest when using 4 threads.

The slightly higher-than-ideal speedup and efficiency can be attributed to reduced memory contention and improved cache utilization in the parallel implementation, which allows threads to perform more work per unit time than predicted by a simple linear model.
Overall, the optimal number of threads for CSR multiplication is 4.

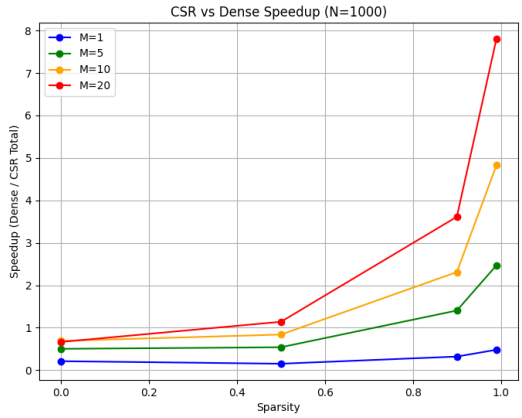
Having established that the optimal number of threads is 4 for both CSR construction and CSR multiplication, we proceed to evaluate the parallel implementation with 4 threads and compare it against dense multiplication. A few important considerations are:

- The number of iterations is critical, as it highlights the trends for CSR construction alone, as well as CSR construction combined with multiple multiplications.
- The comparison should be made between *CSR construction + CSR multiplications* and *dense multiplication*.

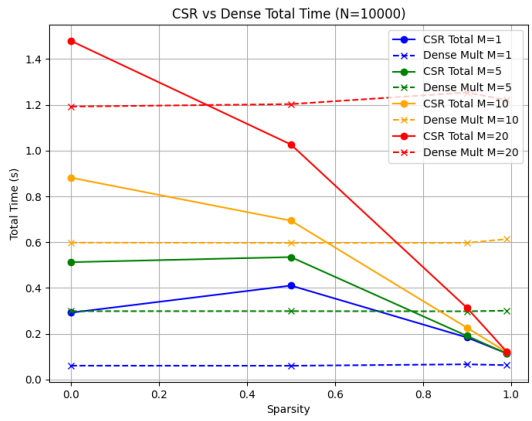
Figures below present the execution times and speedup of CSR and dense parallel implementations for varying numbers of iterations and sparsity levels, for $N = 10^3$ and $N = 10^4$.



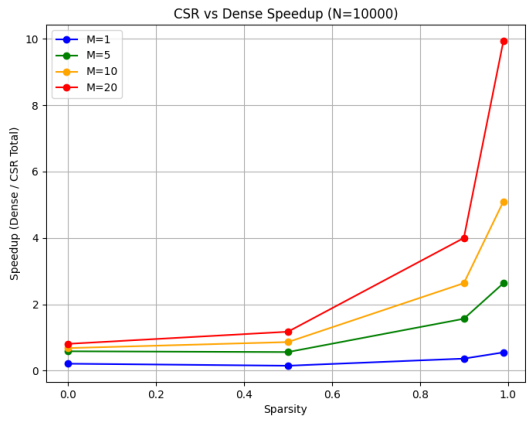
(a) Total Time vs Sparsity (CSR Total vs Dense, varying M)



(b) Speedup vs Sparsity (CSR Total / Dense, varying M)



(a) Total Time vs Sparsity (CSR Total vs Dense, varying M)



(b) Speedup vs Sparsity (CSR Total / Dense, varying M)

- For $M = 1$, dense multiplication is more efficient regardless of sparsity. This is because the overhead of converting the matrix to CSR format dominates the total execution time. This effect is evident in the time plots, where the dense curve lies below the CSR curve, and in the speedup plots, where the speedup is below 1, indicating a slowdown rather than acceleration.
- For $M = 5$, CSR outperforms dense multiplication for sparsity greater than approximately 0.8.
- For $M = 10$, CSR becomes faster when sparsity exceeds roughly 0.6.

- For $M = 20$, CSR outperforms dense multiplication even for sparsity values above approximately 0.4.

These results indicate that as the workload increases, CSR multiplication becomes more advantageous, outperforming dense multiplication even at lower sparsity levels.

Additionally, the speedup plots show that both increasing the workload and increasing sparsity contribute to higher speedup compared to dense multiplication, highlighting the efficiency of CSR for larger, sparser problems.

Exercise 2.3

A task-based parallel Merge Sort was implemented using OpenMP. The algorithm exploits the divide-and-conquer nature of Merge Sort by assigning recursive subarray sorting to independent OpenMP tasks. Synchronization is enforced with `#pragma omp taskwait` to ensure correct merging. To minimize task-management overhead, a threshold of 1000 elements is used, below which sorting proceeds sequentially.

Algorithm 5 Parallel Merge Sort (Task-Based)

```
procedure ParallelMergeSort(arr, left, right)
    if left < right then
        mid ← left + (right − left)/2
        if (right − left) < THRESHOLD then
            SerialMergeSort(arr, left, right)
        else
            spawn task ParallelMergeSort(arr, left, mid)
            spawn task ParallelMergeSort(arr, mid + 1, right)
            taskwait                                        ▷ Synchronization point
            n1 ← mid − left + 1
            n2 ← right − mid
            Allocate L[0..n1 − 1] and R[0..n2 − 1]
            Copy arr[left..mid] to L
            Copy arr[mid + 1..right] to R
            i ← 0, j ← 0, k ← left
            while i < n1 and j < n2 do
                if L[i] ≤ R[j] then
                    arr[k] ← L[i], i ← i + 1
                else
                    arr[k] ← R[j], j ← j + 1
                end if
                k ← k + 1
            end while
            Copy remaining elements of L or R to arr
            Free L and R
        end if
    end if
end procedure
```

Analysis of Results

The algorithm was evaluated on datasets of size $N = 10^7$ and $N = 10^8$ integers, using between 2 and 16 threads.

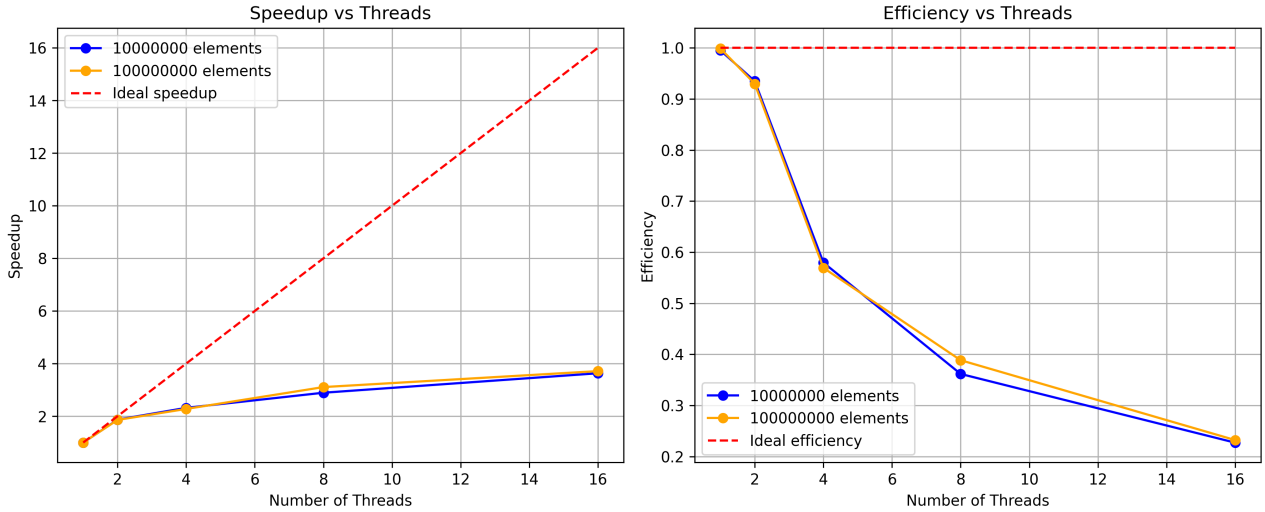


Figure 8: Speedup and Efficiency vs. Number of Threads

Figure 8 illustrates the impact of increasing thread counts on both speedup and efficiency. The following observations summarize the main performance trends:

- The speedup increases with the number of threads but remains **sub-linear**, reaching only about **3.7x** at 16 threads instead of the ideal 16x.
- Performance gains flatten beyond 4 threads, indicating diminishing returns from adding more cores.
- Efficiency drops sharply as thread count increases, from approximately 0.95 at 2 threads to about **0.23** at 16 threads, showing underutilization of CPU resources.

These scaling limitations can be attributed to algorithmic, runtime, and hardware-related factors, summarized below:

- **Task Scheduling Overhead:** The threshold of 1000 is relatively low for a dataset of 10^8. This generates an excessive number of fine-grained tasks, causing the OpenMP runtime to spend significant cycles on task creation and scheduling rather than actual sorting.
- **Allocator Contention:** Frequent `malloc`/`free` calls within parallel regions force threads to contend for memory allocator locks. This effectively serializes the execution whenever multiple threads attempt to allocate memory simultaneously.
- **Bandwidth Saturation:** Merge Sort is inherently memory-bound ($O(N)$ data movement). As more threads operate concurrently, they saturate the memory bandwidth, stalling execution as cores wait for data.

Overall, the task-based OpenMP implementation achieves a speedup of approximately **3.7x** on 16 threads. However, scalability is constrained by the combined effects of task granularity overhead, memory allocator contention, and bandwidth saturation.